

Learn Low-Level Design in 25 Days: Day 25—Final Recap: Building Your Own LLD Portfolio for…

The Developer’s Dilemma

TechPreneur

Published November 7, 2025

Free: No

Read on Freedium · original

Contents

Why Build an LLD Portfolio? 0

What to Include in Your Portfolio 0

1. Pick 5–7 Case Studies 0

2. Structure Each Project Like a Mini Design Doc 0

3. Visuals Matter: Add Diagrams 0

4. Reflect on Design Trade-offs 0

5. Host It Professionally 0

How to Talk About Your Portfolio in Interviews 0

Try It Yourself 0

Summary 0

You've finally made it through 24 intense days of design drills. You've built everything from a chat system to a food delivery app, practiced modeling caches, and even debated the eternal trade-off between performance and readability.

Now imagine this: you walk into an interview room (or log into Zoom). The interviewer says, "Let's design a parking lot system."

Your mind doesn't panic anymore. You start sketching — identify the core entities, talk about responsibilities, relationships, and patterns. You're not just solving a problem; you're **communicating design like an engineer.**



That's where we are today — **Day 25 of Learn Low-Level Design in 25 Days.** Today's goal is to help you **package all your learning into a professional LLD portfolio** — something that showcases your depth, clarity, and craft to interviewers.

By the end of this post, you'll know:

- How to turn your daily practice systems into portfolio projects

- How to document your designs effectively
- How to discuss trade-offs and decisions like a senior developer

Why Build an LLD Portfolio?

Let's be honest — "I can code" isn't enough anymore. At top tech interviews, companies want to see how you **think in systems**, not just syntax.

An **LLD portfolio** helps you stand out because it:

- Demonstrates real-world modeling skills
- Proves you understand trade-offs (not just textbook patterns)
- Makes your interviews structured, confident, and visual

Think of it like a designer's portfolio — but instead of mockups, you're showcasing **class diagrams, flows, and reasoning.**

What to Include in Your Portfolio

Let's turn your 25-day journey into something tangible.

1. Pick 5–7 Case Studies

From the series, choose the ones that represent variety and depth:

- Chat System (Concurrency, Real-time updates)
- Food Delivery App (Complex object interactions)
- Movie Ticket Booking (State and availability management)
- Cache System (LRU/LFU logic)
- Parking Lot (Classic OOD problem)

Each of these covers different **design patterns, object relationships, and trade-offs.**

2. Structure Each Project Like a Mini Design Doc

A clean structure makes your thinking easy to follow:

```
# System Name: Movie Ticket Booking

## 1. Problem Statement
Design a movie ticket booking system that supports seat selection, payment, and multiple theaters.

## 2. Key Requirements
- Users can browse movies and book seats
- Theater managers can add shows
- Prevent double-booking

## 3. Core Classes
- Movie, Show, Seat, Booking, Payment, Theater, User

## 4. Relationships Diagram
User ---> Booking ---> Seat ---> Show ---> Movie

## 5. Key Design Patterns
- Singleton for DatabaseConnection
- State for Seat (Available, Reserved, Booked)
- Strategy for Payment methods

## 6. Trade-offs
- Why use an enum over subclassing for Seat state
- How to balance real-time consistency vs. performance

## 7. Extensions
- Notifications
- Refund handling
```

This kind of documentation mirrors **how senior engineers present system design proposals** internally — concise, logical, and visual.

3. Visuals Matter: Add Diagrams

You don't need fancy tools. Use **PlantUML**, **draw.io**, or even ASCII blocks:

```
User ---> Booking ---> Seat ---> Show ---> Movie
```

Or a simplified class diagram:

```
Class: Seat
  + seatId
  + status
  + reserve()
  + release()

Class: Booking
  + bookingId
  + seats[]
  + makePayment()
```

When interviewers see visuals, it instantly conveys clarity of thought.

4. Reflect on Design Trade-offs

Every good portfolio includes why you made a choice — not just what you designed.

Examples:

- Why did you pick a **Strategy pattern** over simple if-else for payment handling?
- How did you ensure **thread safety** in concurrent systems like Chat or Cache?
- What's the **scalability limit** of your Parking Lot design?

Interviewers love these insights because they show **engineering maturity** — not just coding skill.

5. Host It Professionally

You can use:

- **GitHub + Markdown** — clean, simple, searchable
- **Notion or Medium** — great for visual storytelling
- **LinkedIn Articles** — to share your learning journey

If you publish publicly, use this format:

```
Project Name – [Design Patterns Used]
Short Summary (2–3 lines)
GitHub/Notion Link
Key Learning
```

Example:

> **Movie Ticket Booking — State, Strategy, Singleton**_ Designed a modular system for booking, payments, and seat management. Focused on thread-safe seat locking and extensible payment methods._

How to Talk About Your Portfolio in Interviews

Here's a quick 3-step framework for discussion:

1. **Start with context**: "I built a simplified version of a movie ticket system to explore concurrency and seat management."
2. **Describe your approach**: "I used the State pattern to handle seat transitions and Singleton for DB access."
3. **Reflect on trade-offs**: "I considered Observer for real-time updates but skipped it due to complexity for this version."

That's exactly how strong candidates stand out — they show both **clarity and curiosity.**

Try It Yourself

Challenge 1: Pick any one design from the past 24 days. Write a one-page summary using the 7-section format above.

Challenge 2: Create a GitHub repo titled "LLD Case Studies". Add your designs as markdown files with diagrams and notes.

Bonus: Share the repo on LinkedIn with your reflections — it helps you internalize your learning and attract opportunities.

Summary

Key takeaways from Day 25:

- A well-structured **LLD portfolio** demonstrates thought clarity and design depth
- Focus on **core classes, relationships, patterns, and trade-offs**
- Visuals and documentation amplify your impact
- Be ready to **explain reasoning, not just code**

You've now completed all 25 days — from core principles to real-world systems. But more importantly, you've built the mindset of a **system thinker**, not just a coder.

If you want to revisit or save the full series, here's the complete list: Learn Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)

Bookmark it, practice it, and keep designing better systems — one class diagram at a time.